**Objetivos da aula:**

- Compreender como aplicar testes unitários em aplicações desenvolvidas com NestJS;
- Identificar o que deve ser testado em **DTOs, controllers e services**;
- Entender o papel de **mocks** na substituição de dependências;
- Diferenciar, na prática, os conceitos de **mock, stub e spy**;
- Analisar e interpretar arquivos de teste (*.spec.ts).

**Repositório:** https://github.com/arleysouza/tpII-2026-1-testes-unitarios-nestjs

**Índice:**

i. Fundamentos de Testes Unitários no NestJS
ii. Testes Unitários no NestJS

## i. Fundamentos de Testes Unitários no NestJS

### 1.1. Introdução

Nas aulas anteriores, dois pontos fundamentais foram desenvolvidos separadamente.

Primeiro, foi apresentado o NestJS como um framework que impõe organização estrutural à aplicação, separando claramente responsabilidades entre entrada HTTP, lógica de negócio e acesso a dados. Em seguida, foram estudados os testes unitários, entendidos como mecanismos automáticos para verificar se pequenas partes do sistema continuam funcionando corretamente após alterações no código.

Nesta aula, esses dois temas se unem.

O objetivo agora não é apenas compreender o que são testes unitários, mas entender **como testá-los em uma aplicação NestJS**, respeitando a arquitetura do framework e aproveitando seus recursos, especialmente a **injeção de dependência**. Em aplicações reais, controllers e services não são funções isoladas e soltas: eles dependem de outras classes, serviços externos, banco de dados, repositórios e componentes de infraestrutura. Por isso, testar em NestJS exige compreender não apenas o comportamento esperado, mas também **como isolar dependências** para verificar uma unidade específica de forma controlada.

Em termos práticos, testar no NestJS significa responder perguntas como:

- o service aplica corretamente a regra de negócio?
- o controller chama o service da maneira esperada?
- uma exceção é lançada quando um dado inválido é recebido?
- a unidade testada funciona mesmo quando suas dependências são simuladas?

Essa é a transição natural entre a Aula 8, que apresentou os princípios gerais dos testes com Jest, e o contexto arquitetural do NestJS, no qual tais testes passam a ser aplicados sobre classes, módulos e dependências injetadas.

### 1.2. O que significa testar uma unidade em NestJS

Na definição clássica, um teste unitário verifica a menor unidade testável do sistema, normalmente uma função, um método ou um pequeno módulo. No contexto do NestJS, essa ideia continua válida, mas precisa ser interpretada de acordo com a estrutura do framework.

Em NestJS, as unidades mais comuns de teste são:

1. **Services**
    - concentram a lógica de negócio
    - aplicam regras do sistema
    - coordenam acesso a dependências
2. **Controllers**
    - recebem dados da requisição
    - chamam os services
    - retornam a resposta
3. **Providers auxiliares**
    - classes específicas de validação, autenticação, utilitários ou adaptação

Na prática, o **service** costuma ser o foco principal dos testes unitários, porque é nele que se concentram as decisões do sistema. Isso está alinhado com a própria arquitetura do NestJS, em que o controller deve apenas receber a requisição e delegar a lógica ao service.

Exemplo conceitual:

- **UsersController** recebe GET /users/:id
- chama UsersService.findOne(id)
- o **UsersService** consulta o banco
- se o usuário não existir, lança NotFoundException

Nesse fluxo, diferentes níveis de teste são possíveis:

- testar o **controller** para verificar se ele repassa corretamente o id ao service;
- testar o **service** para verificar se ele retorna o usuário correto;
- testar o **service** para verificar se lança exceção quando o usuário não existe.

Portanto, ao testar uma unidade em NestJS, o objetivo não é testar a aplicação inteira de uma vez, mas sim isolar uma classe ou método e verificar seu comportamento sob condições controladas.

### 1.3. Por que a arquitetura do NestJS favorece testes

Um dos motivos pelos quais o NestJS se destaca em aplicações backend é sua organização explícita e padronizada. A separação entre **controller**, **service** e **module**, aliada ao uso de **injeção de dependência**, reduz o acoplamento entre componentes.

Essa característica é especialmente importante para testes.

Na Aula 8, foi destacado que um bom teste unitário deve ser:

- isolado;
- determinístico;
- simples;
- independente.

Essas qualidades ficam muito mais fáceis de alcançar quando a aplicação já foi construída com baixo acoplamento. No NestJS, em vez de uma classe criar manualmente suas dependências, o framework as injeta automaticamente por meio do construtor. Isso traz uma consequência direta para os testes: **é possível substituir a dependência real por uma dependência simulada**.

Exemplo conceitual:

```
@Injectable()
export class UsersService {
  constructor(private readonly databaseService: DatabaseService) {}
}
```

Nesse caso, o `UsersService` depende de `DatabaseService`. Em produção, essa dependência fornece acesso real ao banco. Em um teste unitário, porém, não queremos usar o banco real, pois isso tornaria o teste mais lento, mais frágil e menos isolado. Em vez disso, podemos fornecer um objeto falso, controlado pelo teste.

Logo, a arquitetura do NestJS favorece testes porque:

- separa bem as responsabilidades;
- evita que classes criem dependências manualmente;
- permite substituir dependências por versões simuladas;
- torna a lógica mais fácil de verificar em isolamento.

Em outras palavras, a **testabilidade** não surge apenas no momento de escrever os testes; ela já começa no modo como o sistema foi arquitetado.

### 1.4. Papel da injeção de dependência nos testes

A injeção de dependência é um dos conceitos centrais do NestJS e, no contexto de testes unitários, ela se torna ainda mais relevante.

Sem injeção de dependência, uma classe poderia estar escrita assim:

```
export class UsersService {
  private readonly databaseService = new DatabaseService();
}
```

Esse tipo de implementação cria um problema sério: a dependência está rigidamente acoplada à classe. Como consequência, o teste não consegue substituí-la facilmente por uma versão controlada.

No NestJS, o padrão correto é outro:

```
export class UsersService {
```

```
    constructor(private readonly databaseService: DatabaseService) {}
  }
```

Agora o `UsersService` não cria a dependência; ele apenas a recebe. Isso significa que, no teste, podemos fornecer um objeto simulado no lugar do serviço real.

Esse ponto se conecta diretamente ao conteúdo da Aula 8 sobre **stubs, mocks e spies**, que são justamente mecanismos usados para substituir ou observar dependências em testes unitários.

Em termos conceituais:

- **stub**: controla o valor retornado por uma dependência;
- **mock**: verifica se a dependência foi chamada corretamente;
- **spy**: observa uma função real sem substituí-la totalmente.

No NestJS, isso aparece com frequência em cenários como:

- simular o retorno de um repositório;
- verificar se um service chamou determinado método;
- observar se um método de outra classe foi acionado com os parâmetros corretos.

Assim, a injeção de dependência não é apenas um recurso de arquitetura. Ela também é o mecanismo que torna o NestJS naturalmente compatível com boas práticas de testes unitários.

### 1.5. O que deve e o que não deve ser testado em uma unidade

Ao começar a testar aplicações NestJS, um erro comum é tentar verificar coisas demais dentro do mesmo teste. A Aula 8 já destacou uma regra importante: **um teste deve validar apenas um comportamento**.

No contexto do NestJS, isso significa que o teste unitário deve se concentrar na unidade em análise, e não em todo o ambiente ao redor.

**O que testar**

Em um **service**, normalmente devem ser testados:

- retornos esperados para entradas válidas;
- tratamento de casos de borda;
- lançamento de exceções;
- interação correta com dependências.

Em um **controller**, normalmente devem ser testados:

- encaminhamento correto de parâmetros;
- chamada do método adequado do service;
- retorno esperado com base na resposta do service.

**O que evitar em um teste unitário**

Um teste unitário não deve depender de:

- banco de dados real;

- API externa real;
- data e hora do sistema sem controle;
- comportamento de outras unidades não simuladas.

Isso se relaciona diretamente ao princípio do **isolamento**, enfatizado na Aula 8: se o teste depende de banco, rede ou múltiplos módulos externos, deixa de ser verdadeiramente unitário.

Exemplo de raciocínio incorreto:

- testar `UsersService.findOne(id)` consultando o banco real;
- depender de dados previamente inseridos;
- verificar também o comportamento do controller no mesmo teste.

Esse tipo de abordagem dificulta identificar a origem de uma falha. Se algo der errado, não saberemos se o problema está:

- na lógica do service;
- no acesso ao banco;
- na configuração do ambiente;
- nos dados cadastrados.

Por isso, a pergunta correta ao elaborar um teste unitário em NestJS é:

> *"Qual comportamento específico desta unidade quero verificar?"*

Essa pergunta orienta o recorte adequado do teste.

### 1.6. Services como foco principal dos testes unitários

Em aplicações NestJS, o ponto mais importante para testes unitários costuma ser o **service**, porque é nele que se concentram as regras de negócio. A Aula 7 já caracteriza o service como o componente responsável por implementar regras, acessar banco e tratar dados.

Considere, por exemplo, um método como este:

```
async findOne(id: number) {
  const [user] = await this.databaseService.db
    .select()
    .from(users)
    .where(eq(users.idUser, id))
    .limit(1);

  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado.`);
  }

  return user;
}
```

Esse padrão aparece claramente na Aula 7, em que o service consulta o banco e lança `NotFoundException` quando não encontra o registro.

Em termos de testes unitários, há pelo menos dois cenários relevantes:

1. Caso de sucesso
    - quando o usuário existe, o método deve retornar o objeto correspondente.
2. Caso de erro
    - quando o usuário não existe, o método deve lançar `NotFoundException`.

Observe que, para testar isso de forma unitária, não precisamos de um banco real. Precisamos apenas controlar o comportamento da dependência responsável pela consulta.

Esse modelo é especialmente didático porque evidencia três ideias ao mesmo tempo:

- a regra de negócio está no service;
- a dependência externa pode ser simulada;
- os testes devem cobrir tanto fluxo principal quanto cenário de erro, como já defendido na Aula 8.

### 1.7. Controllers também podem ser testados

Embora o service seja geralmente a prioridade, o controller também pode ser alvo de testes unitários. No NestJS, o controller é responsável por receber a requisição, extrair dados e retornar a resposta.

Exemplo conceitual:

```
@Controller("api/users")
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  findAll() {
    return this.usersService.findAll();
  }
}
```

Nesse tipo de caso, o controller contém pouca lógica própria. Mesmo assim, ele pode ser testado para verificar:

- se o método existe e pode ser chamado;
- se repassa corretamente o parâmetro ao service;
- se retorna o mesmo valor obtido do service.

O foco aqui não é validar HTTP real, decorators ou integração com o framework inteiro. O foco é simplesmente verificar o comportamento da classe controller como unidade isolada.

Exemplo de raciocínio de teste:

- dado um `UsersService` simulado;
- quando `controller.findOne(1)` for chamado;
- então o controller deve chamar `usersService.findOne(1)`.

Esse tipo de teste costuma ser mais simples que o teste de service, pois o controller normalmente delega quase tudo ao service. Ainda assim, ele é útil para reforçar a separação de responsabilidades e a ideia de que cada camada pode ser verificada independentemente.

### 1.8. Testar exceções em aplicações NestJS

Outro ponto essencial em testes unitários com NestJS é a verificação de exceções. A Aula 7 mostrou que o framework oferece exceptions prontas, como NotFoundException, BadRequestException e ConflictException, cada uma associada a um status HTTP específico.

Isso é importante porque muitas regras de negócio não se limitam a retornar valores; elas também precisam garantir que erros sejam tratados de forma clara e padronizada.

Exemplos típicos:

- usuário não encontrado → NotFoundException
- dado inválido → BadRequestException
- conflito de unicidade → ConflictException

Do ponto de vista dos testes, isso significa que não basta verificar apenas os cenários de sucesso. É necessário testar também se a unidade:

- falha quando deve falhar;
- lança o tipo correto de exceção;
- comunica adequadamente a situação de erro.

Em uma aplicação NestJS bem estruturada, testar exceções é tão importante quanto testar retornos válidos, porque o comportamento correto do sistema inclui também sua forma de responder a entradas inválidas e situações anômalas.

### 1.9. Relação entre Jest e NestJS

A Aula 8 apresentou o Jest como framework de testes no ecossistema JavaScript/TypeScript, oferecendo execução de testes, asserções, mocks e relatórios. No NestJS, o Jest é normalmente a ferramenta padrão para testes unitários.

A relação entre os dois pode ser entendida assim:

- o NestJS fornece a arquitetura da aplicação;
- o Jest fornece os mecanismos para testar essa arquitetura.

Assim, ao testar uma classe NestJS, continuamos usando os mesmos elementos centrais já estudados:

- describe(...) para agrupar testes;
- test(...) ou it(...) para definir cenários;
- expect(...) para verificar resultados;
- toBe, toEqual, toThrow, resolves, rejects e outros matchers conforme o caso.

A diferença não está na sintaxe do Jest, mas no **tipo de objeto que passa a ser testado**. Em vez de funções isoladas como `somar` ou `calcularMedia`, passamos a testar:

- métodos de services;
- métodos de controllers;
- interações entre classes e dependências simuladas.

Em outras palavras, o Jest continua sendo a ferramenta de teste; o que muda é o contexto arquitetural no qual ele é aplicado.

## ii. Testes Unitários no NestJS

### 2.1. O que está sendo testado em cada tipo de arquivo .spec.ts

No repositório, há três grupos principais de testes:

- testes de **DTO**
- testes de **controller**
- testes de **service**

A diferença entre eles não é apenas organizacional. Cada grupo testa **uma unidade diferente** e, por isso, também **moca dependências diferentes** — ou, em alguns casos, **não moca nada**.

A forma correta de ler cada teste é sempre esta:

- **unidade sob teste**: qual classe ou função está sendo avaliada;
- **dependências mockadas**: o que foi substituído para isolar a unidade;
- **comportamento verificado**: qual regra ou responsabilidade o teste quer garantir.

Essa leitura evita um erro muito comum ao olharmos para um jest.fn() e entendemos que "o teste inteiro é sobre o mock", quando na verdade o mock é apenas um mecanismo de isolamento.

### 2.2. Testes de DTO

**O que está sendo testado**

Nos testes de DTO, a unidade sob teste é o próprio DTO, isto é:

- CreateUserDto
- UpdateUserDto
- CreateCarDto
- UpdateCarDto

Aqui, o objetivo não é testar controller, service ou banco.

O objetivo é verificar se os decorators de validação e transformação estão funcionando corretamente.

Exemplo do projeto:

- CreateUserDto valida nome mínimo e formato do e-mail
- CreateCarDto transforma a placa para maiúsculas e valida o padrão Mercosul

**O que é mockado**

**Nada.**

Esse ponto precisa ficar muito claro: em testes de DTO, normalmente não se mocka service, banco, controller ou rota HTTP. O teste trabalha diretamente com a classe DTO, convertendo um objeto simples em instância e executando as validações.

Exemplo real:

```
const dto = plainToInstance(UpdateCarDto, {
  plate: 'bbb2b22',
});

await expect(validate(dto)).resolves.toHaveLength(0);
```

Esse padrão aparece em cars.dto.spec.ts .

**O que exatamente está sendo verificado**

No caso acima, o teste verifica duas coisas:

1. o valor informado é aceito como válido;
2. a transformação e a validação do DTO estão corretas.

Há também um teste explícito para a transformação:

```
const dto = plainToInstance(CreateCarDto, {
  idUser: 1,
  plate: 'abc1d23',
});

expect(dto.plate).toBe('ABC1D23');
```

Esse teste verifica especificamente que o `@Transform(...)` converte a placa para maiúsculas.

```
@Transform(({ value }) =>
  typeof value === 'string' ? value.trim().toUpperCase() : value,
)
```

**Conclusão sobre DTO**

Portanto, em testes de DTO:

- **unidade sob teste**: o DTO;
- **mockado**: nada;
- **o que se verifica**: validação e transformação.

### 2.3. Testes de Controller (delegação)

**O que está sendo testado**

Nos testes de controller, a unidade sob teste é o controller:

- UsersController
- CarsController

O controller, no projeto, não concentra regra de negócio. Sua responsabilidade é:

- receber parâmetros;
- receber DTOs;
- chamar o método correto do service;
- devolver o resultado.

Exemplo do código real:

```
@Get()
findAll() {
  return this.usersService.findAll();
}
```

Esse método está em `users.controller.ts`.

Logo, quando testamos o controller, **não estamos testando a lógica do service**. Estamos testando se o controller delega corretamente.

**O que é mockado**

Nos testes de controller, quem é mockado é o **service**.

Exemplo do arquivo `users.controller.spec.ts`:

```
beforeEach(() => {
  usersService = {
    create: jest.fn(),
    findAll: jest.fn(),
    findOne: jest.fn(),
    update: jest.fn(),
    remove: jest.fn(),
  };

  controller = new UsersController(usersService as unknown as UsersService);
});
```

Aqui, o que está acontecendo é o seguinte:

- a unidade sob teste é `UsersController`;
- a dependência dele é `UsersService`;
- em vez de usar o `UsersService` real, o teste cria um objeto falso;
- cada método desse objeto falso é um `jest.fn()`.

Ou seja:

- **quem está sendo testado**: `UsersController`
- **quem está sendo mockado**: `UsersService`

**O que exatamente `jest.fn()` está fazendo**

Quando escrevemos:

```
findAll: jest.fn()
```

não estamos testando `findAll` do service real. Estamos criando uma função falsa para ocupar o lugar dessa dependência.

Essa função falsa pode ser usada de três formas:

1. **como mock**, porque substitui a dependência real;
2. **como stub**, quando definimos o que ela deve retornar;
3. **como spy**, quando verificamos se ela foi chamada.

Essa distinção aparece com muita clareza no seguinte teste:

```
it('delega a listagem ao service', async () => {
  const response = [{ idUser: 1, name: 'Maria', email: null }];
  usersService.findAll.mockResolvedValue(response);

  await controller.findAll();
  expect(usersService.findAll).toHaveBeenCalled();
});
```

**O que está sendo testado nesse exemplo**

Unidade sob teste

```
controller.findAll()
```

Dependência mockada

```
usersService.findAll
```

**O que o teste configura**

```
usersService.findAll.mockResolvedValue(response);
```

Aqui o teste diz:

"Quando o controller chamar `usersService.findAll()`, essa função falsa deve resolver com response."

Portanto, o retorno não vem do banco, nem do service real. O retorno foi programado pelo próprio teste.

**O que a asserção verifica**

```
await expect(controller.findAll()).resolves.toEqual(response);
```

Essa linha verifica que o controller devolve exatamente o resultado recebido da dependência.

Já esta linha:

```
expect(usersService.findAll).toHaveBeenCalled();
```

verifica que o controller de fato invocou o método do service.

**Conclusão sobre o uso de mock nesse teste**

Nesse único trecho:

- `usersService.findAll` é **mock**, porque substitui a dependência real;
- `mockResolvedValue(response)` faz esse mock atuar como **stub**, pois define o valor de retorno;
- `toHaveBeenCalled()` usa esse mesmo mock como **spy**, pois verifica a chamada.

Então,

o objeto falso criado com `jest.fn()` substitui o método real do service; depois o teste define o que ele retorna e, por fim, verifica se ele foi chamado.

**Conclusão sobre controller**

Em testes de controller:

- **unidade sob teste**: o controller;
- **mockado**: o service;
- **o que se verifica**: delegação, passagem de parâmetros e retorno.

### 2.4. Testes de Service (regras de negócio)

**O que está sendo testado**

Nos testes de service, a unidade sob teste é o service:

- UsersService
- CarsService

Aqui, diferentemente do controller, o foco não é delegação.

O foco é a regra de negócio:

- criação válida;
- busca de entidade inexistente;
- atualização com regras específicas;
- tratamento de conflito;
- tradução de erros de infraestrutura para exceções do NestJS.

Exemplo do service real:

```
async findOne(id: number) {
  const [user] = await this.databaseService.db
    .select()
    .from(users)
    .where(eq(users.idUser, id))
    .limit(1);

  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado.`);
  }

  return user;
}
```

Nesse caso, quando testamos `UsersService.findOne`, queremos verificar:

- se ele retorna o usuário quando existe;
- se lança `NotFoundException` quando não existe.

**O que é mockado**

Nos testes de service, quem é mockado é o **acesso ao banco**, representado por `DatabaseService`.

Exemplo do arquivo `users.service.spec.ts`:

```
beforeEach(() => {
  db = {
    select: jest.fn(),
    insert: jest.fn(),
    update: jest.fn(),
    delete: jest.fn(),
  };

  service = new UsersService({ db } as unknown as DatabaseService);
});
```

Aqui:

- **quem está sendo testado**: `UsersService`
- **quem está sendo mockado**: `DatabaseService`, mais especificamente o objeto `db`

Isso significa que o service real é executado, mas o banco real não participa.

Esse ponto é decisivo:

> em testes de service, a lógica do service é real; o que é falso é apenas a infraestrutura.

**Como o banco é mockado**

Como o código usa Drizzle com chamadas encadeadas, o teste precisa simular essa API.

Por isso existem funções auxiliares como:

```
function createSelectChain<T>(result: T) {
  return {
    from: jest.fn().mockReturnThis(),
    where: jest.fn().mockReturnThis(),
    limit: jest.fn().mockResolvedValue(result),
    orderBy: jest.fn().mockResolvedValue(result),
    innerJoin: jest.fn().mockReturnThis(),
  };
}
```

Essa função cria um objeto falso que se comporta como uma consulta do Drizzle.

Assim, quando o service executa:

```
this.databaseService.db.select().from(...).where(...).limit(1)
```

o teste consegue controlar exatamente o resultado final.

**Exemplo 1 — teste de sucesso**

```
it('retorna o usuário criado quando o nome ainda não existe', async () => {
  const dto = { name: 'Maria', email: 'maria@example.com' };
  const createdUser = { idUser: 1, name: 'Maria', email: 'maria@example.com' };

  db.select.mockReturnValueOnce(createSelectChain([]));
```

```
    const insertQuery = createInsertChain([createdUser]);
    db.insert.mockReturnValue(insertQuery.chain);

    await expect(service.create(dto)).resolves.toEqual(createdUser);
  });
```

Vamos separar isso corretamente.

**Unidade sob teste**

```
service.create(dto)
```

**Dependência mockada**

- db.select
- db.insert

**O que o mock está simulando**

- db.select.mockReturnValueOnce(createSelectChain([])) simula que não existe usuário duplicado;
- db.insert.mockReturnValue(insertQuery.chain) simula a operação de inserção;
- createInsertChain([createdUser]) configura o valor que será retornado na inserção.

**O que está sendo verificado**

A asserção:

```
await expect(service.create(dto)).resolves.toEqual(createdUser);
```

não está verificando se "o mock funciona".

Ela está verificando que **o `UsersService.create` devolve o usuário criado** quando a verificação de unicidade passa e a inserção ocorre com sucesso.

Ou seja, quem está sendo testado é a lógica do service, não o banco mockado.

**Exemplo 2 — teste de dados enviados ao banco**

```
await service.create(dto);

expect(insertQuery.values).toHaveBeenCalledWith({
  name: 'Maria',
  email: 'maria@example.com',
});
```

Aqui, o teste está verificando outra responsabilidade:

- além de criar o usuário, o service montou corretamente os dados enviados ao insert.

Portanto:

- **unidade sob teste**: `UsersService.create`
- **dependência mockada**: `db.insert(...).values(...)`
- **o que se verifica**: os dados passados ao banco

**Exemplo 3 — teste de exceção**

```
await expect(service.findOne(99)).rejects.toMatchObject({
  name: NotFoundException.name,
  message: 'Usuário 99 não encontrado.',
});
```

Nesse caso:

- **unidade sob teste**: `UsersService.findOne`
- **dependência mockada**: `db.select`
- **simulação feita pelo mock**: consulta retorna lista vazia
- **o que se verifica**: o service lança a exceção correta

Esse ponto é muito importante:

> o teste não verifica "se o banco encontrou algo". O teste verifica que **o service reage corretamente ao cenário em que o banco não encontrou nada**.

**Exemplo 4 — tradução de erro de banco**

```
const duplicateError = createDatabaseError('23505', 'duplicate key');
const insertQuery = createInsertChain([], duplicateError);
db.insert.mockReturnValue(insertQuery.chain);

await expect(service.create(dto)).rejects.toMatchObject({
  name: ConflictException.name,
  message: 'Já existe um usuário com esse nome.',
});
```

Aqui a ideia é ainda mais específica:

- o banco falso foi configurado para lançar erro `23505`;
- o teste verifica que o service converte esse erro em `ConflictException`.

Portanto:

- **o mock simula falha da infraestrutura**
- **a unidade testada é a regra de tradução implementada no service**

**Conclusão sobre service**

Em testes de service:

- **unidade sob teste**: o service;
- **mockado**: o acesso ao banco;
- **o que se verifica**: regra de negócio, exceções e interação com a infraestrutura.

### 2.5. Como ler corretamente um arquivo `.spec.ts`

Um arquivo `.spec.ts` deve ser lido sempre em três blocos.

**1. Preparação do cenário**

Normalmente em `beforeEach`

```
beforeEach(() => {
  usersService = {
    create: jest.fn(),
    findAll: jest.fn(),
    findOne: jest.fn(),
    update: jest.fn(),
    remove: jest.fn(),
  };

  controller = new UsersController(usersService as unknown as UsersService);
});
```

Aqui o teste monta a unidade e suas dependências falsas.

**2. Configuração do comportamento dos mocks**

Exemplo:

```
usersService.findAll.mockResolvedValue(response);
```

ou

```
db.select.mockReturnValueOnce(createSelectChain([]));
```

Nesse ponto o teste diz como a dependência mockada deve se comportar.

**3. Execução + asserção**

Exemplo:

```
await expect(controller.findAll()).resolves.toEqual(response);
expect(usersService.findAll).toHaveBeenCalled();
```

ou

```
await expect(service.findOne(99)).rejects.toMatchObject({
  name: NotFoundException.name,
});
```

Aqui finalmente se verifica a responsabilidade da unidade sob teste.

**Exercícios**

**Exercício 1 –** Duplicidade detectada antes do insert

Adicione no arquivo src/users/users.service.spec.ts um teste para o caso em que o nome do usuário já existe na consulta prévia feita por ensureUniqueUserName. O db.select deve retornar um usuário com o mesmo nome, e o teste deve validar que:

- service.create(dto) lança ConflictException;
- db.insert não é chamado.

Objetivo didático: diferenciar "conflito detectado pela regra de negócio" de "conflito detectado pelo banco com erro 23505".

**Exercício 2 -** Atualização de carro com idUser inexistente

Atualmente no arquivo src/cars/cars.service.spec.ts há teste para criação de carro com usuário inexistente, mas não para atualização. Adicionar um teste em que:

- o carro existe;
- o aluno tenta atualizar idUser para um usuário que não existe;
- service.update(id, { idUser: 99 }) deve lançar NotFoundException;
- db.update não deve ser chamado.

Objetivo: reforçar regra de negócio em fluxo de atualização e verificar bloqueio antes da persistência.

**Exercício 3 -** Remoção de usuário inexistente

No arquivo src/users/users.service.spec.ts há teste para falha por chave estrangeira ao remover usuário com carros, mas falta o cenário de usuário inexistente. Criar um teste em que o delete(...).returning() devolve array vazio e validar que:

- service.remove(50) lança NotFoundException;
- a mensagem é `Usuário 50 não encontrado`.

Objetivo: distinguir falha por inexistência da entidade de falha por integridade referencial.